

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 01 -

PRÉ-PROCESSAMENTO DE TEXTOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS	17

O QUE VEM POR AÍ?

Como fazer a máquina e o ser humano conversarem em uma mesma língua? Esse é o desafio do Natural Language Processing (NLP) ou Processamento de Linguagem Natural (PLN). O PLN é uma subárea da inteligência artificial que se concentra na interação entre computadores e seres humanos por meio da linguagem natural.

Seu objetivo é capacitar as máquinas a compreenderem, interpretar e gerarem informação na mesma linguagem do ser humano. Assim, o PLN combina computação, linguística e aprendizado de máquina para processar e analisar grandes quantidades de dados de linguagem natural.

As aplicações são inúmeras, como chatbots, assistentes virtuais, sistemas de recomendação, análise de sentimentos, tradução automática, sumarização de documentos, extração de informações relevantes etc.

Nessa aula, você vai aprender as principais técnicas de pré-processamento de textos para otimizar o aprendizado de máquina.

HANDS ON

Você já deve saber a importância da qualidade dos dados de treino para o resultado do modelo treinado. Com o PLN, não é diferente.

Vamos fazer um exercício: comece pensando somente na linguagem portuguesa. Você pode se comunicar em diversos dialetos, você pode escrever corretamente ou escolher escrever de forma mais informal, como no caso da internet e de chats. Por exemplo, pense nas formas de escrever a palavra “você”: você, vc, voce, VOCÊ, VC, VOCE, Você e assim por diante...

Toda essa diversidade de formas de escrita precisa ser homogeneizada, de modo que palavras escritas de maneira diferente possam ser identificadas como a mesma palavra. Assim, nessa aula, você vai aprender diversas técnicas para pré-processar textos e melhorar a qualidade dos dados de treino do seu modelo.

SAIBA MAIS

A representação e o processamento de texto em Processamento de Linguagem Natural (PLN) envolvem várias etapas e técnicas que transformam a linguagem natural em um formato que pode ser analisado por algoritmos de machine learning e outras metodologias de inteligência artificial.

Entendendo a tokenização

Tokenização é o processo de dividir um texto em unidades menores chamadas "tokens". Esses tokens podem ser palavras, caracteres ou subpalavras, dependendo da aplicação e do método de tokenização utilizado. A tokenização é uma etapa crucial no PLN porque a maioria dos algoritmos de análise de texto requer a divisão do texto em partes menores e mais gerenciáveis.

Tipos de Tokenização

Vamos agora explorar os tipos de tokenização. Em todos os exemplos, estaremos tokenizando o seguinte texto:

```
texto = "Olá, mundo! Bem-vindos ao processamento de linguagem natural."
```

1. **Tokenização por Caractere (Character Tokenization):** divide o texto em caracteres individuais.

```
tokens = list(texto)

print(tokens)
```

Saída:

```
['O', 'l', 'á', ',', ' ', 'm', 'u', 'n', 'd', 'o', '!', ' ', 'B', 'e', 'm', '-', 'v', 'i', 'n', 'd', 'o', 's', ' ', 'a', 'o', ' ', 'p', 'r', 'o', 'c', 'e', 's', 's', 'a', 'm', 'e', 'n', 't', 'o', ' ', 'd', 'e', ' ', 'l', 'i', 'n', 'g', 'u', 'a', 'g', 'e', 'm', ' ', 'n', 'a', 't', 'u', 'r', 'a', 'l', '.']
```

Esse é o método mais simples de tokenizar. No entanto, ele ignora a estrutura das palavras e da sentença. Esse método ajuda quando temos muitos erros ortográficos e palavras raras, mas é difícil extrair estruturas linguísticas dessa

abordagem. Isso exigiria muitos dados, recursos computacionais e memória, o que faz com que raramente seja utilizado.

2. **Tokenização por Palavra (Word Tokenization):** divide o texto em palavras individuais. Para isso, usaremos uma das bibliotecas python mais comuns em PLN: [nltk \(natural language toolkit\)](#).

```
import nltk
from nltk import word_tokenize
nltk.download('punkt')

tokens = word_tokenize(texto)
print(tokens)
```

Saída:

```
['Olá', ',', 'mundo', '!', 'Bem-vindos', 'ao',  
'processamento', 'de', 'linguagem', 'natural', '.']
```

Esse método de tokenização já é mais interessante do que a tokenização por caractere, visto que as palavras já estão formuladas, reduzindo a complexidade de treinamento do modelo.

Percebam que a função “word_tokenize” do nltk já foi, inclusive, inteligente a ponto de separar a pontuação em tokens próprios. Muitas vezes a tokenização por palavra é somente uma separação usando o espaço, o que faz com que crie um token para “natural.” que fica separado de um token “natural”, por exemplo, aumentando ainda mais o vocabulário e não sendo eficiente.

É fácil entender como o vocabulário poderia crescer muito, aumentando a dimensionalidade do problema, quando consideramos conjugações de verbos, erros ortográficos e outras variações. Por isso, quando usamos essa abordagem de tokenização precisamos nos ater à homogeneização de texto que será mostrada a seguir.

3. **Tokenização por Subpalavra (Subword Tokenization):** divide o texto em subpalavras ou fragmentos de palavras; é útil para lidar com palavras desconhecidas e morfologia complexa.

Essa abordagem busca manter palavras frequentes como entidades únicas, mas dividir palavras raras em unidades menores, conseguindo lidar com complexidade e erros ortográficos.

A tokenização por subpalavra é comumente usada com modelos de deep learning como BERT e GPT, que serão estudados posteriormente.

Um exemplo popular de tokenizador de subpalavra é o Byte Pair Encoding (BPE).

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
tokens = tokenizer.tokenize(texto)
print(tokens)
```

Saída:

```
['ol', '##á', ',', ' ', 'mundo', '!', 'bem', '-', 'vindo', '##s', 'ao', 'processamento', 'de', 'linguagem', 'natural', '.']
```

Podemos ver que o plural, por exemplo, foi separado. Dessa forma, “bem-vindo” e “bem-vindos” ainda seriam agrupados e manteríamos a informação de plural com o token “##s”. Da mesma forma, a tokenização por subpalavra separa conjugações diferentes de verbos, prefixos e sufixos, mantendo o token raiz e os demais. Dessa forma, homogeneizamos sem perder informação. A seguir, veremos técnicas de homogeneização de texto que podemos utilizar, mas nas quais ocorre perda de informação.

A escolha do método de tokenização depende da aplicação específica e dos requisitos do modelo de PLN. A tokenização por palavra é adequada para muitas tarefas de PLN básicas, enquanto a tokenização por subpalavra é mais eficaz para modelos de deep learning que precisam lidar com uma ampla variedade de palavras e morfologias.

Homogeneização de dados

Seguindo naquele mesmo exemplo da tokenização, vamos começar a homogeneizá-lo. A ideia é gerar um corpus mínimo de vocabulário, no qual palavras que expressem a mesma coisa sejam atribuídas ao mesmo token.

Transformação texto para minúsculo

Não queremos que a mesma palavra tenha tokens diferentes somente por causa de uma letra maiúscula, certo? Precisamos homogeneizar o texto para que isso não aconteça, o que é um cenário bem comum em textos maiores.

```
texto = texto.lower()
print(texto)
```

Saída:

```
'olá, mundo! bem-vindos ao processamento de linguagem
natural.'
```

Remoção de pontuação

Objetivo: eliminar caracteres que não possuem relevância para o seu problema em questão.

```
import re
texto = re.sub(r'^\w\s', '', texto)
print(texto)
```

Saída:

```
'olá mundo bemvindos ao processamento de linguagem natural'
```

Remoção de acentuação

Para homogeneizar o texto, precisamos remover a acentuação; dessa forma, se houver variações no texto, não teremos problema.

Para isso, usaremos a biblioteca Unicode, que converte caracteres em seus equivalentes mais próximos em ASCII. É simples e direta, mas pode não lidar com todos os caracteres de todas as línguas perfeitamente.

```
from unidecode import unidecode
texto = unidecode(texto)
print(texto)
```

Saída:

```
'ola mundo bemvindos ao processamento de linguagem natural'
```


Remoção de stopwords

Remoção de palavras comuns que não contribuem significativamente para a análise (como "a", "e", "o"). Para isso, voltaremos a usar a biblioteca `nltk`, que já tem mapeada uma vasta lista de stopwords em português.

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('portuguese'))
print(stop_words)
```

Saída:

```
{'estiveram', 'para', 'esses', 'é', 'fomos', 'nosso',
'esteja', 'teria', 'você', 'nossa', 'numa', 'se', 'tivemos',
'um', 'nossos', 'sem', 'fôssemos', 'fosse', 'houveriam', 'aos',
'seria', 'isso', 'lhe', 'como', 'minha', 'foi', 'os', 'hajam',
'com', 'meus', 'temos', 'elas', 'as', 'tivera', 'estão',
'hajamos', 'houvesse', 'delas', 'estava', 'seu', 'sou',
'tivermos', 'eles', 'nas', 'num', 'forem', 'minhas', 'vocês',
'houvermos', 'há', 'estivera', 'dos', 'mesmo', 'estiver', 'por',
'teríamos', 'foram', 'esteve', 'tem', 'haver', 'tiveram', 'uma',
'houvêramos', 'aquela', 'lhes', 'seremos', 'vos', 'houver',
'houveríamos', 'dela', 'muito', 'de', 'e', 'nem', 'for', 'tém',
'o', 'esta', 'haja', 'fôramos', 'no', 'terão', 'esse',
'teremos', 'aquilo', 'depois', 'estivéssemos', 'pela', 'isto',
'tenho', 'ser', 'nossas', 'estavam', 'tivêramos', 'mais',
'tivéssemos', 'pelos', 'terei', 'tu', 'houveram', 'às', 'ou',
'hei', 'eu', 'a', 'tenham', 'havemos', 'houveremos', 'estes',
'aquelas', 'me', 'estou', 'tinham', 'houvessem', 'éramos',
'meu', 'estejam', 'aquele', 'teve', 'seríamos', 'estivêramos',
'do', 'houvera', 'tinha', 'pelas', 'aqueles', 'quando', 'qual',
'era', 'estivessem', 'estive', 'houverá', 'houvemos', 'essas',
'estamos', 'mas', 'tiver', 'estivesse', 'eram', 'fora', 'dele',
'suas', 'seja', 'teriam', 'seriam', 'estávamos', 'houverem',
'serão', 'que', 'somos', 'fui', 'teu', 'ela', 'entre', 'à',
'das', 'estivermos', 'da', 'houve', 'pelo', 'são', 'seus',
'tínhamos', 'será', 'tua', 'em', 'estar', 'sejam', 'hão', 'te',
'teus', 'não', 'houverei', 'nós', 'estiverem', 'fossem', 'este',
'estivemos', 'já', 'ele', 'formos', 'estejamos', 'na', 'serei',
'só', 'tenhamos', 'tiverem', 'também', 'terá', 'tivessem',
'houvéssemos', 'nos', 'houverão', 'sua', 'quem', 'houveria',
'tivesse', 'deles', 'está', 'estas', 'tuas', 'sejamos', 'tenha',
'ao', 'até', 'tive', 'essa'}
```

Podemos ver que já temos várias stopwords mapeadas dentro da biblioteca nltk, mesmo na língua portuguesa. Podemos estender ela para nosso caso adicionando manualmente mais palavras, caso necessário, ou removendo da lista palavras que sejam relevantes para o contexto do modelo.

```
tokens = word_tokenize(texto)
valid_tokens = [token for token in tokens if token not in
stop_words]
texto = ' '.join(valid_tokens)
print(texto)
```

Saída:

```
'ola mundo bemvidos processamento linguagem natural'
```

Stemming

Stemming é o processo de reduzir as palavras às suas raízes ou radicais, o que pode resultar em palavras não reconhecíveis, removendo sufixos e prefixos. Frequentemente o stemming resulta em formas truncadas que podem não ser palavras reais (e.g., "computador" -> "comput"). Para isso, usaremos a biblioteca NLTK. O NLTK possui um stemmer chamado RSLPStemmer, que é especificamente projetado para lidar com a língua portuguesa.

```
import nltk
nltk.download('rslp')
from nltk.stem import RSLPStemmer
from nltk.tokenize import word_tokenize

stemmer = RSLPStemmer()
tokens = word_tokenize(texto)
stemmed_tokens = [stemmer.stem(token) for token in tokens]
texto = ' '.join(stemmed_tokens)
print(texto)
```

Saída:

```
'ola mund bemv process lingu natur'
```

Lematização

Reduz as palavras à sua forma canônica ou dicionária, o que geralmente produz resultados mais precisos e legíveis do que apenas cortar sufixos e prefixos como no stemming, além de manter sua integridade semântica (e.g., "computadores"

-> "computador"). Então, utilizaremos a biblioteca Spacy, visto que a biblioteca NLTK não possui suporte nativo para lematização em português.

Para isso, você deve, além de instalar a biblioteca spacy `pip install spacy`, também baixar o modelo python -m spacy download pt_core_news_sm (Portuguese · spaCy Models Documentation).

```
import spacy

nlp = spacy.load('pt_core_news_sm')
doc = nlp(texto)
token_lematizado = [token.lemma_ for token in doc]
texto = ' '.join(token_lematizado)
print(texto_lematizado)
```

Saída:

```
'ola mundo bemvinr processamento linguagem natural'
```

Correspondência fuzzy

Para corrigir erros de digitação, variações ortográficas, abreviações ou outros tipos de variações, podemos usar técnicas de correspondência fuzzy. Ela permite encontrar correspondências aproximadas entre textos levando em consideração a semelhança entre eles, mesmo que não sejam idênticos.

Além disso, a correspondência fuzzy também é frequentemente usada em correção automática de palavras, correspondência de registros em bancos de dados e em sistemas de pesquisa para fornecer sugestões de consulta, deduplicação de dados e detecção de plágio, entre outros.

Existem várias técnicas para a correspondência fuzzy, entre elas: similaridade de Jaro, similaridade de Jaro-Winkler, distância de Levenshtein e distância de Damerau-Levenshtein. Vamos entendê-las melhor?

Similaridade de Jaro

O resultado do algoritmo Jaro é um valor entre 0 e 1, em que 0 indica nenhuma similaridade e 1 indica uma correspondência perfeita. Quanto maior o valor, maior a similaridade entre as duas strings.

$$d_j = \frac{m}{3a} + \frac{m}{3b} + \frac{m-t}{3m}$$

Em que:

- m é o número de correlações entre caracteres.
- a e b são os tamanhos dos textos a serem comparados.
- t é o número de transposições.

Similaridade de Jaro-Winkler

Este algoritmo estende a Similaridade de Jaro adicionando uma constante que favorece quando a comparação ocorre com textos que possuem equivalência longa no início.

$$sim_w = sim_j + lp(1 * sim_j)$$

Em que:

- sim_j é a similaridade de Jaro entre os dois textos.
- l é o tamanho do prefixo comum entre os dois textos até no máximo 4 caracteres.
- p é uma constante de ajuste e pode ser lida como o peso que você deseja dar ao fator do prefixo comum l . Essa constante não deve exceder 0,25, visto que o máximo de l é 4; caso contrário, a similaridade poderia dar um número maior que 1. O valor padrão usado é 0,1.

Distância de Levenshtein

Também conhecida como distância de edição, é uma medida da quantidade mínima de operações necessárias para transformar uma string em outra. As operações permitidas são inserção, exclusão e substituição de um caractere. Assim, a distância de Levenshtein entre duas strings é o número mínimo de operações necessárias para transformar uma string na outra.

Por exemplo:

- A distância de Levenshtein entre "casa" e "cabra" é 2, pois é necessário substituir o "s" por "b" e adicionar um "r" no final.
- A distância de Levenshtein entre "gato" e "gata" é 1, pois é necessário apenas adicionar um "a" no final da primeira string.

Distância de Damerau-Levenshtein

Estende a distância de Levenshtein ao adicionar a transposição adjacente entre as possíveis operações. Por exemplo: a distância entre as palavras “flom” e “molf” com transposição adjacente é 1, já que somente transpondo o “m” e o “f” conseguimos transformar uma palavra em outra. Na distância de Levenshtein, teríamos o valor de 2: precisaríamos de uma remoção e uma adição.

Vamos comparar essas métricas de similaridade? Para isso, usaremos a biblioteca python jellyfish. Iremos comparar os textos: “aprendizado de máquina” com “aprendizad d maquina” e com “bom dia alunos FIAP”.

```
import jellyfish

texto_original = "aprendizado de máquina"
texto_erro = "aprendizad d maquina"
texto_aleatorio = "bom dia alunos FIAP"

similaridades = [
    "jaro_similarity",
    "damerau_levenshtein_distance",
    "levenshtein_distance",
    "jaro_winkler_similarity"
]

comparacoes = {
    'erro_digitacao': [texto_original, texto_erro],
    'comp_aleatoria': [texto_original, texto_aleatorio]
}

resultados = {}
for distancia in similaridades:
    resultados[distancia] = {
        comp: getattr(jellyfish, distancia)(
            comparacoes.get(comp)[0], comparacoes.get(comp)[1]
        )
        for comp in comparacoes
    }
resultados
```

Saída:

```
{
  'jaro_similarity': {
    'erro_digitacao': 0.9027910685805423,
    'comp_aleatoria': 0.5164805954279639
  }
}
```

```
},
'damerau_levenshtein_distance': {
  'erro_digitacao': 3,
  'comp_aleatoria': 19
},
'levenshtein_distance': {
  'erro_digitacao': 3,
  'comp_aleatoria': 19
},
'jaro_winkler_similarity': {
  'erro_digitacao': 0.9416746411483253,
  'comp_aleatoria': 0.5164805954279639
}
}
```

Podemos ver que a distância de Damerau-Levenshtein e a distância de Levenshtein deram os mesmos valores, visto que não tivemos transposições em nenhuma das comparações. As diferenças foram de remoção (a letra “o” de “aprendizado” e a letra “e” de “de”) ou substituição (á por a) na comparação de erro de digitação e quase tudo na comparação aleatória.

Já a similaridade de Jaro-Winkler retornou um valor maior para a comparação com erro de digitação do que a similaridade de Jaro. Isso aconteceu porque nossa comparação possui uma semelhante longa no prefixo, como esperado, e deu o mesmo valor para a comparação aleatória.

Dado um vocabulário de frequência, você pode usar esses algoritmos para corrigir seu texto, como faz a biblioteca [Sympell](#). Só tenha cuidado: caso esteja usando seus dados para construção do vocabulário, construa ele usando somente os dados de treino e nunca os de teste, a fim de não ocorrer vazamento de dados.

Podemos pensar em vários outros pré-processamentos que serão necessários dependendo do caso, como tratamento de URL e remoção de números, entre outros. É importante destacar que nem sempre se indica todos os pré-processamentos: você deverá entender caso a caso o que é necessário para o contexto do seu modelo e realizar experimentos para definir o que faz mais sentido para o resultado dele.

A tokenização escolhida também ajuda na definição das homogeneizações necessárias. Por exemplo, a tokenização por palavra exige bem mais homogeneizações do que a tokenização por subpalavra. Conforme vimos, várias das homogeneizações mencionadas fazem com que percamos informação: stemming e

lematização cortam parte da informação; a tokenização por subpalavra permite manter essa informação intacta e não necessitar dessa homogeneização.

Além disso, às vezes a pontuação também pode ser relevante. Tudo depende do problema a ser modelado e você vai precisar experimentar para entender os trade-offs entre as homogeneizações, o resultado do modelo e os recursos necessários.

Vale alertar que quando há grande quantidade de dados, o que é comum em tarefas de processamento de linguagem natural, você precisará de bastante recurso computacional para o pré-processamento, o qual poderá demorar bastante.

Limitar o vocabulário é uma alternativa interessante para reduzir custos computacionais e otimizar o processo, mesmo que percamos informação. Uma forma de fazer isso é substituir os tokens raros para “desconhecido”, por exemplo.

Além disso, utilizar ambientes paralelizáveis, como Spark, e ambientes em Cloud, como Google Dataflow, AWS Glue, Azure Data Factory e outros, é bastante recomendado quando lidamos com grandes quantidades de dados.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu os tipos mais comuns de pré-processamento de textos para utilização em algoritmos de processamento de linguagem natural. Você já consegue limpar, homogeneizar, corrigir e tokenizar. Lembre-se que a forma como você pré-processa o dado tem impacto direto na qualidade do seu modelo.

É aconselhado experimentar várias das técnicas apresentadas nessa aula e entender o impacto para o resultado do modelo. Lembrando que a escolha do pré-processamento ideal depende do modelo, dos recursos computacionais disponíveis e do nível de precisão desejado.

REFERÊNCIAS

BIRD, S.; KLEIN, E.; LOPER, E. **Natural Language Processing with Python**. 1. ed. [s.l.]: O'Reilly Media, Inc., 2009.

NLTK. **Documentation**. 2024. Disponível em: <<https://www.nltk.org>>. Acesso em: 30 jul. 2024.

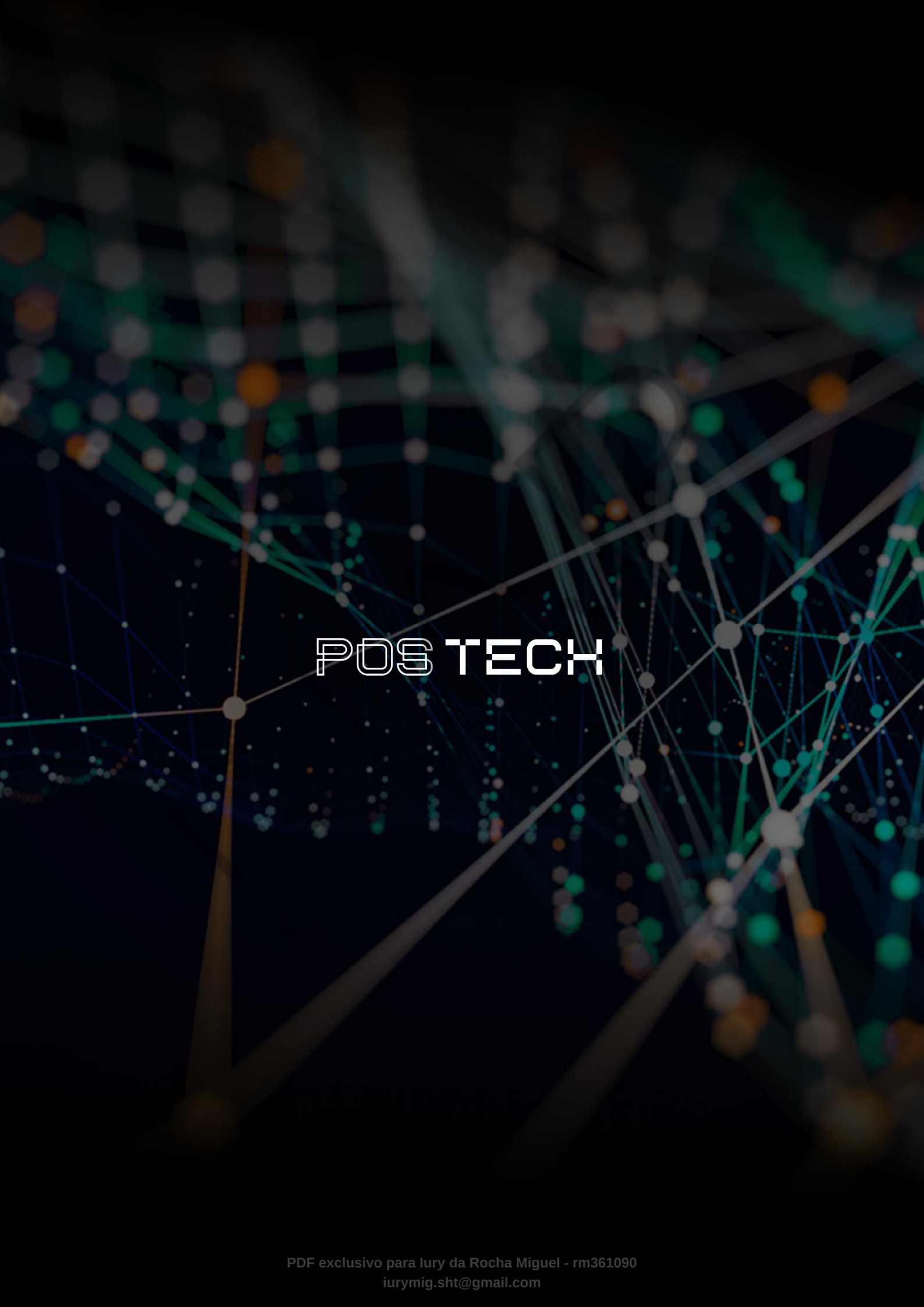
TUNSTALL, L.; VON WERRA, L.; WOLF, T. **Natural Language Processing with Transformers**. Revised Edition. [s.l.]: O'Reilly Media, Inc., 2022.

WOLFGARBE. **SymSpell**. 2024. Disponível em: <<https://github.com/wolfgarbe/SymSpell>>. Acesso em: 30 jul. 2024.

PALAVRAS-CHAVE

Processamento De Linguagem Natural. Aprendizado de Máquina. Limpeza de Dados. Tokenização.

EMENDAS



POSTECH